

CONNECTIFY BACKEND

Detailed Development Summary & Technical Report

[Authentication](#) → [Database](#) → [Validation](#) → [Middleware](#) → [JWT](#) → [Profiles](#) → [Current Progress](#)

Technical handover report documenting the backend work completed so far.

1. Executive Summary

This report documents the Connectify backend work from the authentication foundation through the current profile-related work. The backend uses Node.js, Express, TypeScript, MongoDB/Mongoose, Zod, bcrypt, JWT and dotenv. The work focused on creating a structured, maintainable backend rather than putting all logic inside route handlers.

The project was organized around configuration, database connectivity, schemas, middleware, controllers, models, routes, shared types and logging. This separation allows collaborators to extend the backend without mixing validation, authentication, database access and business logic.

The development process also involved real debugging, including duplicate-email registration handling, authentication failures and the observed POST /api/v1/auth/login 403 response. After authentication, work progressed into authenticated profile saving and synchronization with the frontend authentication state.

2. Backend Architecture

The main request pattern is:

```
Client → Route → Validation → Authentication Middleware → Controller/Service → Mongoose Model → MongoDB → Response
```

Routes identify the endpoint. Validation checks incoming data. Middleware performs reusable request-level work such as authentication. Controllers/services perform application logic. Models handle persistence. Configuration and core utilities keep infrastructure concerns separate.

3. Environment Configuration

dotenv is used to load environment variables, while Zod validates them before they are exported for application use. This keeps values such as the MongoDB URI, port and JWT secret out of hard-coded business logic and prevents invalid configuration from silently reaching runtime.

A representative structure is:

```
const environmentSchema = z.object({
  PORT: z.coerce.number(),
  MONGO_URI: z.string().min(1),
  JWT_SECRET: z.string().min(1)
})

export const env = environmentSchema.parse(process.env)
```

The important idea is fail-fast configuration: if a required environment value is missing or invalid, the application detects it early.

4. Database Connection

A dedicated connectDatabase function was created using Mongoose. It reads the validated MongoDB URI from the environment module, attempts the connection, logs success or failure and rethrows connection errors.

```
import mongoose from 'mongoose'
import { logger } from '../core/logger/logger.js'
import { env } from './env.js'

export async function connectDatabase() {
  try {
    await mongoose.connect(env.MONGO_URI)
    logger.info('Database connected')
  } catch (error) {
    logger.error('Database connection failed', { error })
    throw error
  }
}
```

Separating this function means controllers do not need to manage database connectivity.

5. Logging and Request IDs

Logging and request IDs were used to make API requests traceable. Logs can expose the method, endpoint, status, duration, timestamp and request ID. This was particularly useful when investigating authentication behaviour such as the login 403 response.

6. Express Request Types and JWT Payload

Because authentication middleware attaches user information to Express requests, the project extended `Express.Request` and defined a JWT payload:

```
export interface JWTPayload {
  id: string
  role: string
}

declare global {
  namespace Express {
    interface Request {
      requestId: string
      user?: {
        id: string
      }
    }
  }
}

export {}
```

`JWTPayload` describes the information placed in the JWT. `id` identifies the user and `role` provides the basis for later authorization. `req.user` is optional because public requests do not have an authenticated identity.

7. Registration and Authentication

Registration validates user information, checks whether the email already exists, hashes the password and creates the user. The registration flow is:

- Receive name, email and password.
- Validate the request using the registration schema.
- Check MongoDB for an existing account with the email.
- Return HTTP 409 Conflict when the email is already in use.
- Hash the password with `bcrypt`.
- Create the user with the hashed password.
- Return a successful response without exposing the password.

The 409 response is important because it distinguishes an existing-account conflict from a general server failure.

8. Password Security with `bcrypt`

`bcrypt` was used to ensure raw passwords are not stored in MongoDB. Registration transforms the password into a hash. Login compares the supplied password with that stored hash. The original password does not need to be recovered.

9. Zod Authentication Schemas

The backend includes dedicated validation schemas for registration and email verification. The registration password rules include at least eight characters, an uppercase letter, a lowercase letter, a number and a special character.

```
const registerSchema = z.object({
  email: z.string().email(),
```

```
password: z.string()
  .min(8)
  .regex(/[A-Z]/)
  .regex(/[a-z]/)
  .regex(/[0-9]/)
  .regex(/^[A-Za-z0-9]/)
})
```

A separate `verifyEmailSchema` provides a specific contract for email-verification requests rather than reusing unrelated registration fields.

10. Login and JWT

Login verifies that the account exists, compares the submitted password with the bcrypt hash and establishes authenticated identity through JWT. The JWT payload uses the user id and role.

Authentication answers “Who is this user?” while authorization answers “What is this user allowed to do?”. The authenticated ID becomes especially important for protected user-owned operations.

During development, `POST /api/v1/auth/login` returned a 403 response. The request logs provided a concrete trace for debugging the authentication flow.

11. Protected Routes and Middleware

The protected-route pattern is:

```
Authorization header → Bearer token → JWT verification → decode payload → req.user → controller
```

Once the token is verified, middleware attaches the authenticated user ID to `req.user`. Controllers should use that verified identity rather than trusting a user ID supplied by the browser.

12. API Versioning and Route Organization

Authentication endpoints were organized under the versioned `/api/v1/auth` namespace. The login endpoint was observed as `POST /api/v1/auth/login`. Versioning creates room for future API changes while keeping authentication separate from other feature routes.

13. Transition to Profiles

After authentication, the backend work moved toward profiles. Authentication data identifies and secures the account; profile data represents information belonging to that account within Connectify.

The frontend `AuthProvider` was structured around both User and Profile state. The backend profile flow follows the same conceptual separation and uses the authenticated identity as the ownership boundary.

14. Profile Save Flow

The profile save/update work connects the authentication system to real application data. The expected flow is:

```
Authenticated request → Validate profile body → Read req.user.id → Find/Create profile → Save → Return profile
```

The key security rule is that ownership comes from the verified JWT identity, not an arbitrary user ID sent by the client. This pattern can later support profile editing, preferences, connections and other user-owned features.

15. Models and Data Separation

Mongoose models define how application entities are persisted. The user model handles core account identity and authentication-related information, while profile information can remain a separate data boundary. This prevents profile changes from unnecessarily affecting password or login logic.

16. Error Handling

The backend uses meaningful HTTP status codes. Duplicate registration uses 409 Conflict. Authentication failures were surfaced through responses and logs, including the observed login 403. The project should continue distinguishing validation, authentication, authorization, conflict, not-found and server/database failures.

17. What I Worked On

- Set up and structured the Connectify backend with Node.js, TypeScript and Express.
- Configured environment variables using dotenv and Zod.
- Created the MongoDB/Mongoose connection layer.
- Worked with logging and request IDs for API debugging.
- Defined JWTPayload and extended Express.Request for authenticated user data.
- Created registerSchema and verifyEmailSchema validation.
- Implemented registration and duplicate-email protection.
- Implemented bcrypt password hashing and comparison.
- Built the login/JWT authentication foundation.
- Organized authentication under /api/v1/auth.
- Debugged the POST /api/v1/auth/login 403 response.
- Moved the backend from authentication into authenticated profile functionality.
- Worked on the profile save flow and user/profile ownership relationship.
- Maintained separation between authentication/account data and profile data.

18. Technical Lessons

- Authentication is a chain: registration, hashing, login, JWT creation, verification and protected routes must agree.
- Validation should happen before business logic and database operations.
- Passwords should never be stored as plain text.
- Protected operations should derive ownership from req.user.id.
- Environment validation prevents configuration problems from becoming confusing runtime failures.
- Request IDs and structured logging make debugging easier.
- HTTP status codes should communicate the type of failure.
- TypeScript request declarations should match middleware behaviour.
- Separating authentication and profiles keeps future features maintainable.

19. Current Backend State

Connectify has progressed from a basic server setup into a structured backend application. Authentication provides the security foundation, while profile functionality is the current application layer being connected to that identity. The project now has validated configuration, database connectivity, validation, models, request typing, bcrypt password security, JWT authentication, organized routes, logging and authenticated profile work.

20. Recommended Next Steps

- Complete and test the authenticated profile save/update endpoint end-to-end.

- Ensure profile ownership always comes from req.user.id.
- Complete email verification end-to-end if it is not fully wired yet.
- Standardize authentication and authorization error responses.
- Add automated tests for registration, duplicate email, login, protected routes and profile operations.
- Add appropriate MongoDB indexes/unique constraints, especially for account email.
- Document backend endpoints for frontend collaborators.
- Replace remaining mock API interactions with real backend endpoints as features stabilize.

21. Final Conclusion

The Connectify backend work represents a progression from infrastructure to application functionality. Authentication established the security boundary; Zod established input contracts; MongoDB/Mongoose established persistence; bcrypt established secure password handling; JWT established authenticated identity; Express middleware established reusable request processing; and profile development began turning that identity into user-owned data.

The major achievement is the structure around these features. The backend is now organized so collaborators can continue adding Connectify functionality without mixing configuration, validation, authentication, routing and database responsibilities. This document serves as the technical handover of that work.